旅館（宿泊施設）向け予約フォーム 実装設計

開発者用 ／ 作成 2026-10-23・更新 2026-10-27（v3: 先に作って当日設定する前提で設定項目を汎用化）／ 元ファイル: docs/旅館用予約フォーム検討資料/旅館用予約フォーム_実装設計.md ／ 状態: 実装中（第 1 段階）

1 部屋だけの宿泊施設が LINE / Web の予約フォームから「チェックイン日〜チェックアウト日」で予約を受けられるようにする。**既存の予約フォームに日時選択モード `booking_mode: 'stay'` を追加**する方式。予約テーブル・LIFF 送信・Webhook・CRM・リマインダー・フォロー・Google カレンダー・メール・一覧 / 分析はすべて流用する。**施設の回答を待たずに先に作り、ヒアリング当日は設定画面に入力するだけ**にする。そのため料金・人数・休業・キャンセル料・受付期限・チェックイン時刻は「一覧を店舗が作る」形で持つ。

1. 要望と対応

要望（ヒアリング済み）	対応
部屋は 1 部屋のみ	`room_count`（既定 1）
チェックイン 15:00 以降 / チェックアウト翌 10:00	`check_in_time` / `check_out_time`。選ばせたい場合は `check_in_time_options`
11/1〜11/2 の予約があっても 11/2 15:00 から別の客を受けたい	**「泊（夜）」単位で空きを判定**
チェックイン / アウトの日付を選んで予約。時間表示不要、◯× は欲しい	カレンダーを 2 回タップ。マスに「料金」（= 空き）か「空室なし」
曜日・シーズン別料金、日数分の自動計算	「料金表」を複数持ち、曜日 / 祝前日 / 祝日 / 季節 / 特定日に割り当て。1 泊ずつ決めて合計
人数で料金が変わる（1 名 14,000 / 2 名 20,000 / 3 名 26,000。0〜5 歳無料）	料金表 =「有料人数 → 1 泊料金」。人数の区分 `guest_categories` は店舗が自由に作り、各区分を 有料人数に数える / 無料 / 1 人 1 泊 ◯円加算 から選ぶ
連泊割引	`long_stay_discounts`（N 泊以上で % / 1 泊あたり円 / 合計から円。段階可）
12 か月先まで / 前日の午前中まで / 最大 7 泊	`advance_booking_days: 365` / `min_advance_days: 1` + 条件付き `{ from_time: '12:00', min_days: 2 }` / `max_nights: 7`
LINE と Web の両方	同じ宿泊モードで `form_type` line / web の 2 フォーム。「他フォームにも反映」で同期

キャンセルポリシー（当日・前日 100% / 〜1 週間前 50% / それ以前 無料）	段階式キャンセル料 `cancel_rules.fee_tiers`（日数 + 任意の時刻境界）。規定文の自動生成、LINE キャンセル時に金額を提示して確認 or 施設へ誘導、記録と通知。**請求はしない**

2. 空きの考え方（最重要）

単位は「夜（night）」。日付 `d` の夜 = `d` のチェックイン時刻 〜 `d+1` のチェックアウト時刻。

- 予約 `[checkin, checkout)` は `checkin` 〜 `checkout − 1` の夜を使う
- 夜 `d` が埋まっている = その夜を使う有効予約（`status <> 'cancelled'`）数 + その夜の窓に重なる Google カレンダー予定（店舗カレンダー + `block_calendar_ids`）数 ≥ `room_count`
- 夜 `d` が受けられない = 休業（曜日 / 特定日 / 期間）、祝日不可、受付期間外（`resolveEarliestBookable` の日数より前、`advance_booking_days` より先）
- チェックイン日に選べる = 夜 `d` が空いていて受けられる。チェックアウト日に選べる = `checkin` 〜 `d − 1` の夜が全部空き、泊数が範囲内。**夜 `d` 自体は埋まっていてもよい**（Airbnb と同じ）

日	月	火	水	木	金	土
11/1 空室なし A 宿泊中	11/2 20,000円 A 10:00 退出 B 15:00 イン可	11/3 20,000円	11/4 20,000円	11/5 空室なし カレンダー予定	11/6 20,000円	11/7 26,000円 土曜用の表

3. 設定（型）

```
calendar_settings: { booking_mode?: 'calendar' | 'multiple_dates' | 'stay'; stay_settings?:
StaySettings; }

interface StayGuestCategory {           // 人数の区分（店舗が自由に作る）
  id; label;                            // 「6歳以上」「0〜5歳」「小学生」…
  charge: 'adult' | 'free' | 'fixed_per_night';   // 有料人数に数える / 無料 / 1 人 1 泊 price 円加算
  price?: number; min: number; max: number;
  counts_toward_capacity: boolean;      // 定員に数える（既定 true）
}
interface StayRateTable {               // 料金表
  id; name; mode: 'by_count' | 'per_person';
  prices: { [count: string]: number };  // by_count: "1": 14000, "2": 20000, "3": 26000
  per_person_price?: number;            // per_person
  extra_person_price?: number;          // by_count で表の最大人数超の 1 人あたり加算（無し = 受けない）
}
interface StayPricing {
  rate_tables: StayRateTable[]; default_rate_table_id: string;
  weekday_rate_table_ids?: { [dow: string]: string };
  holiday_eve_rate_table_id?: string; holiday_rate_table_id?: string;
  seasons?: Array<{ id; name; start; end; rate_table_id }>;   // 両端含む。重なりは保存時に拒否
  date_rate_table_ids?: Array<{ date; rate_table_id }>;       // 最優先
  long_stay_discounts?: Array<{ min_nights; type: 'percent' | 'per_night' | 'total'; value }>;
}
interface StaySettings {
  check_in_time; check_out_time; check_in_time_options?: string[];
  min_nights; max_nights; room_count;
  closed_weekdays?; closed_dates?; closed_ranges?: Array<{ start; end; label? }>;
  total_capacity?: number; guest_categories: StayGuestCategory[];
  pricing: StayPricing; show_price_on_calendar: boolean; price_note?: string;
  block_calendar_ids?: string[];        // 空き判定に含める追加の Google カレンダー（他媒体の iCal 購読
など）
}
cancel_rules: {
  deadline_hours; policy_text; show_policy_on_form; notify_store_on_cancel;   // 既存
  fee_tiers?: Array<{ days_before: number; until_time?: string; percent: number }>;
  fee_base?: 'stay' | 'total';                  // 既定 'stay'
  self_cancel_with_fee?: 'confirm' | 'deny';    // 既定 'confirm'
}
```

1 泊の料金の決め方

1. 料金表の選択（上が強い）：`date_rate_table_ids` > `seasons` > `holiday_rate_table_id` > `holiday_eve_rate_table_id` > `weekday_rate_table_ids[dow]` > `default_rate_table_id`
2. 有料人数 = `charge: 'adult'` の区分の合計

3. `by_count` は `prices[有料人数]`（最大人数超は `extra_person_price` × 超過。無ければ `null` = 受けない。表に無い少人数も `null`）。`per_person` は 単価 × 有料人数
4. `fixed_per_night` の区分は `price` × 人数 を加算
5. 合計 = Σ 各夜 − 連泊割引（`min_nights` を満たす最大の段階を 1 つ。`percent` は合計に、`per_night` は泊数 × 値、`total` は値。0 円未満にしない）

キャンセル料の決め方

- `daysBefore` = チェックイン日 − 操作日（JST の日付差。当日 0、前日 1）
- `fee_tiers` を `days_before` の大きい順に見て、最初に当てはまる段階の `percent`。当てはまる = `daysBefore` > `days_before`、または 等しくて（`until_time` 無し または 現在時刻 < `until_time`）。無ければ最後の段階。空なら 0%
- 例 A: `[{7, 0}, {2, 50}, {0, 100}]` = 7 日前以前 無料 / 2〜6 日前 50% / 前日・当日 100%。例 B: `[{7, 0}, {1, until 18:00, 50}, {0, 100}]` = 前日 18:00 まで 50%
- 金額 = percent × 基準額（`fee_base`）。10 円未満切り捨て。既存 `deadline_hours` は先に判定（LINE から一切不可の期限）
- `self_cancel_with_fee = 'confirm'`: 金額を提示 →「キャンセル確定: N」で取り消し。`'deny'`: 金額を案内して施設へ誘導
- 規定文: `policy_text` が空なら `buildCancelPolicyText` が段階表から自動生成

宿泊モードで使わない既存設定	使う既存設定
`business_hours` / `holiday_hours` / `special_business_days` / `blocked_times` / `time_interval` / `extra_minutes` / `multiple_dates_settings` / `allow_exceed_business_hours` / `max_concurrent_events`（→ `room_count`）/ `min_advance_hours` / スタッフの希望日時設定	`advance_booking_days` / `min_advance_days` / `conditional_advance_rules`（日数部分）/ `holidays_as_closed` + `excluded_holiday_types` / `show_customer_*` / `require_customer_*` / `show_message_field` / `show_customer_email` / `notification_email` / `max_concurrent_reservations_per_user` / `cancel_rules`

`normalizeForm()`: `stay_settings` があれば常に `sanitizeStaySettings()`（料金表 0 件なら「基本」を作る。参照先の無い ID は基本に戻す。区分 0 件なら「6歳以上」adult 1〜4 と「0〜5歳」free 0〜3）。`resolveCancelRules` が `fee_tiers` / `fee_base` / `self_cancel_with_fee` も補完。

4. 純粋ロジック（第 1 段階。Vitest 対象）

ファイル	関数

`src/lib/stay-pricing.ts`	`createDefaultStaySettings` / `sanitizeStaySettings` / `paidGuestCount` / `capacityCount` / `validateGuestCounts` / `resolveRateTable(pricing, date, isHoliday)` / `rateTablePrice(table, paidCount)` / `resolveNightPrice(settings, date, counts, isHoliday)` / `resolveLongStayDiscount` / `computeStayTotal(settings, checkin, checkout, counts, isHoliday)` → `{ nights, per_night[], subtotal, discount, total }` / `formatStayRangeJa` / `formatGuestCountsJa`
`src/lib/stay-availability.ts`	`nightsBetween` / `addDays` / `diffDays` / `isNightClosed` / `occupiedCountByNight(range, reservations, events, settings)` / `buildNightStatus(range, occupied, settings, cs, now, isHoliday, isHolidayBlocked, counts)` / `isRangeBookable` / `selectableCheckoutDates`
`src/lib/cancel-rules.ts`（追加）	`sanitizeFeeTiers` / `resolveCancelFee(rules, checkinDate, now, baseAmount)` / `buildCancelPolicyText`
`src/lib/stay-runtime-js.ts`（第 3 段階）	`STAY_RUNTIME_JS`: 同じ判定・料金・規定文の vanilla JS 版（TS と JS の両方を直す）

5. DB（第 2 段階 `20261101000000_add_stay_reservation.sql`）

```
ALTER TABLE reservations
  ADD COLUMN IF NOT EXISTS checkout_date DATE,          -- NULL = 宿泊以外
  ADD COLUMN IF NOT EXISTS stay_nights INTEGER,
  ADD COLUMN IF NOT EXISTS guest_counts JSONB,          -- { "<category_id>": 人数 }
  ADD COLUMN IF NOT EXISTS guest_count INTEGER,         -- 有料人数
  ADD COLUMN IF NOT EXISTS stay_price_total INTEGER,    -- 割引後（サーバー再計算値）
  ADD COLUMN IF NOT EXISTS cancel_fee_percent INTEGER,
  ADD COLUMN IF NOT EXISTS cancel_fee_amount INTEGER,
  ADD COLUMN IF NOT EXISTS cancelled_at TIMESTAMPTZ;
ALTER TABLE reservations ADD CONSTRAINT reservations_checkout_after_checkin
  CHECK (checkout_date IS NULL OR checkout_date > reservation_date);
CREATE INDEX IF NOT EXISTS idx_reservations_stay_range
  ON reservations (store_id, reservation_date, checkout_date) WHERE checkout_date IS NOT NULL;
```

- `reservation_date` = チェックイン日、`reservation_time` = チェックイン時刻（NOT NULL 維持）。**リマインダー 3 種・フォロー・来店履歴は無変更で動く**
- `customer_info` に内訳: `stay_per_night` / `stay_subtotal` / `stay_discount` / `guest_counts` / `guest_labels`
- 二重予約防止: DB 関数 `stay_insert_reservation_checked(p_row JSONB, p_room_count INT)`。`pg_advisory_xact_lock(hashtext(store_id))` → 各夜の有効予約数 ≥ `room_count` なら `RAISE EXCEPTION 'stay_full'` → INSERT。カレンダー側はアプリで直前再確認。Local は JSON で同じ判定

6. API（第 2 段階）

- **新規 `GET /api/stores/{storeId}/stay/availability?form_id=&from=&to=&guests=`**：公開。夜ごとの `{ available, remaining, price }` のみ。`guests` = `{ "<category_id>": n }`。最大 93 日。`reservations` + 店舗カレンダー + `block_calendar_ids` → `buildNightStatus()`。**料金もここで計算**（焼き込みは再デプロイ + 1 時間キャッシュで古くなる）。失敗時 `calendar_degraded: true`
- **`POST /api/reservations`**：追加 `booking_mode: 'stay'` / `checkout_date` / `guest_counts` / `reservation_time`（選択時）。検証: 泊数・受けられない夜・`validateGuestCounts`・料金計算可。`reservation_time` は `check_in_time_options` 以外なら `check_in_time`。料金はサーバー再計算。空き再チェック → DB 関数 → `409 slot_taken`。カレンダー予定 `start = reservation_date T reservation_time` 〜 `end = checkout_date T check_out_time`。来店履歴の `amount` に加算
- **`PATCH /api/reservations/{id}`**：`checkout_date` / `guest_counts` の編集 → 再計算 + `updateReservationEvent`。`cancelled` で `cancel_fee_*` / `cancelled_at`（上書き可）。復元でクリア
- **Webhook**:「キャンセル: N」→ `deadline_hours` → `resolveCancelFee` → `self_cancel_with_fee` に従い確認 / 誘導。「キャンセル確定: N」新設。`parseReservationForm()` に `《チェックイン》` / `《チェックアウト》`。一覧は「11/1(日) 〜 11/4(月) 3泊」

7. 静的 HTML（第 3 段階）

- `renderDateTimeFields()` で `'stay'` のとき `renderStayField()`。時間スロット・スタッフカレンダー判定は出さない
- 人数: `guest_categories` の数だけプルダウン。変更で料金を取り直す。チェックイン時刻: `check_in_time_options` があればプルダウン、無ければ固定表示
- カレンダー: PC 2 か月、スマホ 1 か月 + ‹ ›。マス = 日付 +「¥20,000」/「空室なし」/「−」。選択: 1 回目 = イン、2 回目 = アウト（`selectableCheckoutDates` 以外は新しいイン扱い）。下に「11/1(日) 15:00 → 11/4(月) 10:00 / 3 泊 / 6歳以上 2名・0〜5歳 1名 / ¥60,000 → 連泊割引 −¥6,000 = ¥54,000」
- 空き取得: 開いたとき・月送り・人数変更・送信直前 → `applyStayStatus()`。送信直前に埋まっていたら案内して送信しない
- キャンセルについて: `policy_text` or 自動文。summary: ご宿泊日 / 泊数 / 人数 / 宿泊料金（割引前 → 割引 → 後）。復元機能は日程を復元しない。Web でも同じ UI

```
【希望日時】
《チェックイン》
2026年11月01日（日） 15:00
《チェックアウト》
2026年11月04日（月） 10:00
《泊数》
3泊
《宿泊人数》
6歳以上 2名 / 0〜5歳 1名
《宿泊料金》
¥54,000（連泊割引 -¥6,000 適用。税込・目安）
```

8. 表示側（第 4 段階）

場所	変更
`reservation-detail-items.ts`	チェックイン / チェックアウト / 泊数 / 宿泊人数（区分ごと）/ 宿泊料金（割引の内訳）。キャンセル済みは キャンセル料（admin 行）
予約一覧（3 画面）/ `CustomerDetail.tsx`	「11/1(日) 〜 11/4(月) 3泊」。並び順はチェックイン日
`ReservationEditForm.tsx`	チェックアウト日・人数（区分ごと）の編集。キャンセル時にキャンセル料の計算値（上書き可）
店舗への【予約キャンセル】メール / `reservation-analytics.ts`	キャンセル料の行 / `reservationAmount()` に `stay_price_total`
`email-templates.ts` / リマインダー / フォロー	変更なし

9. 編集画面（第 5 段階）— ヒアリング当日に入力する画面

- `BusinessRulesEditor.tsx` のモードに「宿泊（チェックイン〜チェックアウト）」。宿泊時は時間関係の設定を隠し `StaySettingsEditor.tsx` を表示。予約可能日数・受付開始日・条件付き受付開始・祝日不可・名前 / 電話 / メッセージ / メール・キャンセルルールはそのまま
- `StaySettingsEditor` の構成（担当者用 PDF の設定シートと同じ並び）: ① 基本（時刻 / 時刻選択 / 泊数 / 部屋数 / 料金表示 / 注記）② 人数の区分（行追加 + 定員）③ 休業日（曜日 / 特定日 / 期間）④ 料金表（カード複数。方式・金額・超過加算。基本を 1 つ選ぶ）⑤ 料金表の割り当て（曜日 / 祝前日 / 祝日 / 季節 / 特定日）⑥ 連泊割引 ⑦ 空き判定に含めるカレンダー（既存 `/calendar/list` を流用）⑧ 料金プレビュー
- キャンセルルール設定: 段階（○日前 / ○時まで / ○%）+ 基準 + 有料期間の LINE キャンセル + 規定文プレビュー
- 「他フォームにも反映」で LINE 版と Web 版を同期。プレビューは全日ダミー

- テンプレート「🏨 宿泊（1 部屋）」= `createDefaultStaySettings()`（15:00 / 10:00、1〜7 泊、1 部屋、6歳以上 adult 1〜4・0〜5歳 free 0〜3、料金表「基本」{1: 14000, 2: 20000, 3: 26000}）+ `advance_booking_days: 365` + `min_advance_days: 1` + 条件付き 12:00 + `fee_tiers [{7,0},{2,50},{0,100}]`

10. 設定で吸収できない回答（第 2 弾）

内容	代替 / 時期
部屋タイプ別の在庫	部屋ごとにフォームを分ける。第 2 弾で `stay_settings.calendar_id`
決済・キャンセル料の回収	無し。表示と記録のみ
泊数ごとの固定パック料金	連泊割引で近似。パック方式は第 2 弾
見積もり方式 / 他媒体への双方向同期	無し（他媒体 → こちらは `block_calendar_ids` で対応）

11. テストと作業順序

- `stay-pricing.test.ts`：料金表の優先順位、季節の両端、by_count（表どおり / 超過加算 / 加算なしは null / 少人数 null）、per_person、区分（adult 合算 / free / fixed 加算）、連泊割引（段階・3 方式・0 未満にしない）、内訳と合計、月またぎ、`validateGuestCounts`、`sanitizeStaySettings`
- `stay-availability.test.ts`：`nightsBetween`、チェックアウト日の夜が埋まっていても選べる、途中の夜が埋まると不可、`room_count = 2`、休業（曜日 / 特定日 / 期間）、泊数範囲、終日予定と時刻予定の重なり、受付期間（12:00 境界・365 日）
- `cancel-rules.test.ts`：段階の判定（7 / 6 / 2 / 1 / 0 日前・過ぎた後）、`until_time` 境界、基準と切り捨て、規定文、`resolveCancelRules` の補完

1. **型 + normalizeForm + stay-pricing / stay-availability / cancel-rules + Vitest（今回）**
2. マイグレーション + DB 関数 + POST / PATCH + 空き API（1 日）
3. 静的 HTML + STAY_RUNTIME_JS + LINE メッセージ + Webhook（2〜2.5 日）
4. 表示側（1 日）
5. StaySettingsEditor + キャンセル料の段階 + テンプレート（2 日）
6. staging 手動テスト → CLAUDE.md / API_SPECIFICATION.md

12. ヒアリング当日に設定画面へ入力する項目

担当者用 PDF の「設定シート」と同じ内容。施設の回答はここに書き取って、そのまま画面に入力する。

画面の場所	入力すること	いま分かっている値
基本	チェックイン / アウト時刻、時刻をお客様に選ばせるか、最低・最大泊数、部屋数、料金表示、注記	15:00 / 10:00、固定、1〜7 泊、1 部屋、表示、注記は未定
人数の区分	区分の見出し・扱い・最小最大、定員	6歳以上（有料 1〜4）、0〜5歳（無料 0〜3）。4 名以上と定員は未定
休業日	曜日 / 特定日 / 期間	未定
料金表	基本の表、土曜・祝前日用、繁忙期用 … の金額。4 名以上の加算	基本 = 14,000 / 20,000 / 26,000。他は未定
料金表の割り当て	曜日 / 祝前日 / 祝日 / 季節 / 特定日	未定
連泊割引	何泊以上で何 % または何円	あり（内容未定）
空き判定に含めるカレンダー	他媒体の iCal を購読したカレンダー	他媒体の有無が未定
予約ルール	何日先まで、受付開始日、条件付き（○時以降は翌日不可）	365 日、1 日後、12:00 以降は 2 日後
キャンセルルール	段階（○日前 / ○時まで / ○%）、基準、有料期間の LINE キャンセル	7 日前まで 0% / 2〜6 日前 50% / 前日・当日 100%。6 日前・7 日前の境界と基準は未定